

Lecture 6 Practice Worksheet — Solutions

Problem 1 — Payroll System (Polymorphism): Solution

```
class Employee:
    def __init__(self, name, base_salary):
        self.name = name
        self.base_salary = base_salary

    def monthly_pay(self):
        # base/default pay logic
        return self.base_salary

class Intern(Employee):
    def __init__(self, name, base_salary, stipend):
        super().__init__(name, base_salary)
        self.stipend = stipend

    def monthly_pay(self):
        # Intern gets base salary + stipend
        return self.base_salary + self.stipend

class Contractor(Employee):
    def __init__(self, name, hourly_rate, hours_worked):
        # Contractor may not even have a normal base salary
        super().__init__(name, base_salary=0)
        self.hourly_rate = hourly_rate
        self.hours_worked = hours_worked

    def monthly_pay(self):
        # Contractor is paid purely hourly
        return self.hourly_rate * self.hours_worked

e = Employee("Ali", 80000)
i = Intern("Sara", 20000, 5000)
c = Contractor("Bilal", hourly_rate=2000, hours_worked=35)

print("Employee pay:", e.monthly_pay())
print("Intern pay:", i.monthly_pay())
print("Contractor pay:", c.monthly_pay())
```

Explanation: All three classes (Employee, Intern, Contractor) define the SAME method name `monthly_pay()`. But each class implements it differently. This is polymorphism through method overriding.

Problem 2 — Shopping Cart (Operator Overloading): Solution

```
class ItemPrice:
    def __init__(self, amount, currency):
        self.amount = amount
        self.currency = currency

    def __str__(self):
        return f"{self.currency} {self.amount:.2f}"

    def __add__(self, other):
```

```

        # combine two prices
        if self.currency != other.currency:
            raise ValueError("Currency mismatch")
        return ItemPrice(self.amount + other.amount, self.currency)

    def __mul__(self, qty):
        # price * quantity
        total = self.amount * qty
        return ItemPrice(total, self.currency)

# demo
apple_price = ItemPrice(120, "PKR")
banana_price = ItemPrice(80, "PKR")

cart_total = apple_price + banana_price
line_total = apple_price * 4

print("Cart total:", cart_total)    # PKR 200.00
print("Line total:", line_total)    # PKR 480.00

```

Explanation: `__add__` lets us use `+` on `ItemPrice` objects, and `__mul__` lets us do price * quantity. `__str__` makes printing readable like 'PKR 480.00' instead of .

Problem 3 — Report Generator (Duck Typing): Solution

```

class HtmlReport:
    def generate(self):
        print("Rendering HTML report...")

class TextReport:
    def generate(self):
        print("Generating text report...")

def run_report(report_obj):
    # We don't care what the class is called.
    # We ONLY assume that it has a .generate() method.
    report_obj.generate()

html_r = HtmlReport()
text_r = TextReport()

run_report(html_r)    # Rendering HTML report...
run_report(text_r)    # Generating text report...

```

Explanation: `HtmlReport` and `TextReport` do not inherit from a shared parent class. But both have a `generate()` method. `run_report()` works with any object that has `generate()`. This is Python-style polymorphism via duck typing.